

A THESIS REPORT ON

SMART SCHOOL MANAGEMENT SYSTEM WITH E-LEARNING PORTAL

Integrating AI-Powered Learning Diagnostics, Predictive Performance Analytics, and LLM
Chatbot Assistance

Submitted by:

Lokesh Nargesh

Roll No: MTECH/CS/2024/089

Under the guidance of:

Dr. Rajesh Kumar

Associate Professor, CSE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

UNIVERSITY INSTITUTE OF TECHNOLOGY

JUNE 2026

CERTIFICATE OF APPROVAL

This is to certify that the project report entitled '**Smart School Management System with E-Learning Portal**' submitted by **Lokesh Nargesh** in partial fulfillment of the requirements for the award of the degree of Master of Technology in Computer Science and Engineering is a record of bonafide work carried out by him under our supervision. The results embodied in this thesis have not been submitted to any other University or Institute for the award of any degree or diploma.

Supervisor

Dr. Rajesh Kumar
Associate Professor, CSE

Head of Department

Dr. Amit Sharma
Professor & Head, CSE

CANDIDATE'S DECLARATION

I hereby declare that the work presented in this thesis entitled '**Smart School Management System with E-Learning Portal**' is an authentic record of my own research and development carried out during the period from July 2025 to June 2026 under the guidance of Dr. Rajesh Kumar, Department of Computer Science and Engineering.

I have fully cited and referenced all resources, research articles, and documentation packages used in the execution of this project. No part of this thesis has been copied or duplicated from other student reports, nor has it been submitted elsewhere for academic credit.

Lokesh Nargesh

Date: June 9, 2026

Place: UIT, Campus

ACKNOWLEDGEMENTS

First and foremost, I would like to express my deepest gratitude to my project supervisor, **Dr. Rajesh Kumar**, for his continuous support, guidance, encouragement, and invaluable suggestions throughout the development of this M.Tech project. His insightful critiques and profound technical wisdom kept the project on track and pushed me to achieve enterprise-grade standards in system architecture and AI integration.

I am highly grateful to **Dr. Amit Sharma**, Head of the Computer Science Department, for providing excellent academic resources and laboratories. I also wish to thank my peers and family members for their constant encouragement and backing.

ABSTRACT

The **Smart School Management System with E-Learning Portal** is a comprehensive, multi-role web-based enterprise application designed to automate school operations while introducing modern e-learning features. Built using a robust, decoupled architecture composed of an Angular 19 frontend, a Spring Boot 3.2.5 REST backend, and a MySQL relational database, the system enforces Role-Based Access Control (RBAC) secured via stateless JSON Web Tokens (JWT).

To satisfy final-year M.Tech project standards, this system extends standard CRUD operations by introducing three advanced Artificial Intelligence modules: (1) An **AI Chatbot Helper** integrating Google's Gemini API (with a smart local NLP engine fallback) to assist students with school operations, admissions policies, and syllabi FAQs; (2) An **AI Study Material Recommender** which interrogates student gradebooks, isolates subjects with below-average performance, and surfaces relevant e-learning notes, video lectures, and slides; and (3) An **AI Student Performance Predictor** that implements a predictive diagnostic analyzer using marks distributions and class attendance logs to flag high-risk students and automatically synthesize structured academic improvement plans.

This project successfully bridges school administrative operations with interactive student diagnostic pipelines, demonstrating the applicability of lightweight predictive services in secondary education.

CHAPTER 1: INTRODUCTION & OVERVIEW

1.1 Project Introduction

The educational sector is undergoing a rapid digital transformation. Traditional school environments depend heavily on manual record-keeping and scattered spreadsheets to manage student enrollments, class timetables, attendance trackers, fee registers, and grading registers. This siloed management leads to inefficiencies, administrative overhead, and poor communication among school leaders, faculty members, parents, and students. The integration of administrative operations with academic e-learning portals is crucial to streamlining operations and improving student success.

The **Smart School Management System with E-Learning Portal** represents a unified, cloud-ready solution that addresses these challenges. By incorporating administrative management tools and modern online classrooms into a single, cohesive application, the system enables real-time collaboration. The system accommodates four distinct roles: Administrators (operations management), Teachers (academic delivery), Students (e-learning participation), and Parents (performance tracking).

1.2 Detailed Problem Statement

Existing School Information Systems (SIS) focus primarily on bookkeeping—recording who paid fees, who attended class, and what grades they achieved. They fail to turn this collected data into academic diagnostics. Educators lack automated tools to identify struggling students early, resulting in reactive interventions. Additionally, e-learning content systems (LMS) are typically disconnected from administrative records, leading to double-entry of profiles and course enrollments. There is a need for an integrated, intelligent portal that uses administrative records to generate automated academic interventions.

Moreover, student support mechanisms in current setups are passive. Students must self-diagnose their weaknesses and search for notes or videos. Without a personalized recommender, e-learning content is underutilized, and weak students often fall behind. Parents also lack clear insight into attendance drops or poor test performance until report cards are released, preventing timely support.

1.3 Project Objectives & Goals

The primary objectives of this research and development project are:

1. To design and implement a decoupled, secure REST API backend using Spring Boot and JPA, coupled with a relational MySQL database.
2. To build a responsive, single-page application (SPA) frontend in Angular 19 using standalone components and responsive styles.
3. To implement Role-Based Access Control (RBAC) secured via stateless JWT bearer authentication filters.
4. To construct an automated online MCQ examination engine that grades student responses in real-time and updates gradebooks.
5. To integrate Google's Gemini LLM API with local NLP fallback templates to provide school operations FAQs and chatbot support.
6. To implement a diagnostic recommender system that isolates student academic weaknesses and suggests e-learning notes.
7. To develop a predictive health diagnostic engine that checks student risk factors based on attendance and exam marks, generating tailored remedial checklists.

1.4 Scope and Feasibility Analysis

The scope of this project extends across secondary and higher secondary school setups. We evaluate three areas of feasibility:

- 1. Technical Feasibility:** Spring Boot provides structured API security, JPA enables smooth database operations, and Angular provides reactive dashboard state. The dependencies are open source, confirming technical feasibility. System maintenance requirements are low, as standard JVM and Node runtimes are universally supported.
- 2. Economic Feasibility:** By replacing paper notice sheets, print invoices, physical test cards, and server maintenance with automated, lightweight REST calls, the system reduces overhead costs. Administrative tasks that previously required dedicated staff can now be completed automatically, reducing school operating budgets.
- 3. Operational Feasibility:** System navigation is role-based, ensuring students, teachers, and parents have distinct, tailored dashboards without access confusion. The interface is designed to run efficiently on mobile browsers, making it accessible for parents and students.

CHAPTER 2: LITERATURE REVIEW

2.1 Evolution of School Management Systems

Early school administrative systems (circa 1990s) were localized desktop database applications. These systems were limited to single-terminal installations, requiring office administrators to input registration data manually. Sharing records with teachers or parents required printing reports, leading to paper waste and delays. In the mid-2000s, web-based School Information Systems emerged, utilizing classical Model-View-Controller (MVC) server-side rendering architectures (e.g., PHP, Java Server Pages). While these systems enabled remote access, they suffered from monolithic code bases, high page reload latency, and poor mobile responsiveness.

2.2 Modern Learning Management Systems (LMS)

With the advent of platforms like Moodle, Canvas, and Google Classroom, e-learning became mainstream. These Learning Management Systems provided virtual classrooms, homework submission portals, and discussion boards. However, in secondary education, these platforms are often run separately from the primary administrative database. This segregation forces IT administrators to maintain synchronized records manually, leading to data drift and security vulnerabilities in role management.

2.3 Artificial Intelligence in Education (AIEd)

Research in AIEd has traditionally focused on Intelligent Tutoring Systems (ITS) that customize learning paths. Recently, LLMs like GPT-4 and Gemini have introduced new possibilities for natural language question-answering. However, deploying public LLMs directly in schools raises cost, token overhead, and privacy concerns. This project proposes a hybrid approach: using lightweight, rule-based analytical algorithms for sensitive academic predictions (recommender and performance risk engines) while leveraging a secured, sandboxed Gemini LLM API for non-sensitive student query support. This ensures predictability, data safety, and offline resilience.

CHAPTER 3: SYSTEM ARCHITECTURE & REQUIREMENTS

3.1 Architectural Block Diagrams

The system employs a standard RESTful architecture. The frontend is built as a single-page application in Angular 19, which makes asynchronous HTTP calls to the Spring Boot REST API. The backend processes requests, validates JWT authorization tokens, queries the MySQL database via Hibernate JPA, and returns JSON payloads. File attachments (PDF notes, student submissions) are stored locally or uploaded to cloud storage providers (Cloudinary/Firebase) with URLs saved in the database.

3.2 Architectural Flow Matrix

The system components communicate in the following hierarchy:

Angular Client Side (Port 4200)	Spring Boot API Side (Port 8080)	Data & Services Layer
Auth Service (JWT Storage) Admin/Teacher/Student Views MCQ Quiz Engine AI Chatbot Component	SecurityConfig & JWT Filter REST Controllers Spring Data JPA Services AI Analytics Controller	MySQL Database (20 Tables) Gemini LLM API Client Local File Storage / Uploads

3.3 Software and Hardware Specifications

Development Environment:

- Operating System: Windows 10/11 / Linux (Ubuntu 22.04 LTS)
- Runtime Environment: Node.js (v24.13.0), JDK 17 (Java SE LTS)
- Build Tools: Angular CLI v19, Gradle Wrapper (build.gradle)
- Database: MySQL Community Server (v8.0.32)

Hardware Specifications (Minimum):

- CPU: Intel Core i5 / AMD Ryzen 5, 2.5 GHz Quad-core
- Memory: 8 GB RAM (16 GB Recommended for concurrent compiler runs)
- Disk Space: 5 GB minimum free space for project builds and local file uploads

3.4 Use Case Modeling

The system role mappings translate to specific functional use cases:

1. **Admin Use Case Context:** Accesses student/teacher registers, initiates class-subject associations, publishes bulletins, and audits collections.
2. **Teacher Use Case Context:** Marks attendance logs, schedules notes, uploads homework files, sets quizzes, and reviews gradebooks.
3. **Student Use Case Context:** Browses study folders, submits homework files, starts quizzes, reviews grades, and chats with the AI assistant.
4. **Parent Use Case Context:** Reviews child attendance, results, noticeboard, and fee balances.

CHAPTER 4: DETAILED MODULAR FUNCTIONALITIES

4.1 Description of the 15 Major Modules

- 1. Authentication & RBAC Module:** Manages user login and registration. Implements JWT filters for stateless session tracking. Checks roles (ADMIN, TEACHER, STUDENT, PARENT) to verify authorization before processing REST requests.
- 2. Student Management:** Enables admins to register students, update profile parameters, search profiles by roll number or name, and delete records when needed. Maintains enrollment histories.
- 3. Teacher Management:** Coordinates teacher records, including specialization and date of joining. Supports assigning teachers to subjects and classes.
- 4. Attendance Management:** Enables teachers to record daily attendance. Calculates attendance percentages and flags students below the 75% requirement.
- 5. Class & Section Management:** Coordinates school structures. Admins can create classes and sections to support timetable organization.
- 6. Subject Allocation:** Maps subjects to specific classes and sections. Assigns a primary teacher to deliver content for each subject.
- 7. Timetable Generation:** Schedules subjects throughout the week. Provides students and teachers with their weekly schedules on their dashboards.
- 8. Study Material Module:** Enables teachers to upload e-learning resources (PDF notes, video lecture links, PPTs). Organizes materials by class and subject for students to access.
- 9. Assignment Management:** Enables teachers to create assignments with due dates and grading criteria. Students can submit their work by entering file URLs.
- 10. Online MCQ Quiz Engine:** Automates testing. Teachers create MCQ questions, and students complete the quizzes within time limits. The system grades responses automatically.
- 11. Result & GPA Manager:** Calculates quiz scores and records student performance. Provides a report card display for students and parents.
- 12. Notice Board Module:** Allows admins to publish announcements. Notices are filtered by role (e.g. notices for teachers only vs. general notices).
- 13. Fee Management Portal:** Tracks school fees. Allows admins to register fee payments, record transaction methods, and generate receipts.

14. Library Management System: Manages library books. Tracks book status (available, issued, overdue) and logs borrowing details.

15. Transport Management System: Coordinates school transport routes, bus routes, driver details, and student assignments to transport routes.

CHAPTER 5: DATABASE SCHEMA & DATA DICTIONARY

5.1 Data Dictionary for the 20 Database Tables

The following tables describe the database schemas. Foreign key relationships enforce referential integrity.

Table 5.1: users

Column	Type	Key	Description
id	BIGINT AUTO_INCREMENT	PRIMARY	Unique user identifier
name	VARCHAR(100)	-	User's full name
email	VARCHAR(100)	UNIQUE	Used for system login
password	VARCHAR(255)	-	BCrypt hashed credential
role	VARCHAR(20)	-	ADMIN, TEACHER, STUDENT, PARENT
phone	VARCHAR(20)	-	Contact telephone number
created_at	TIMESTAMP	-	Profile registration timestamp

Table 5.2: classes

Column	Type	Key	Description
class_id	BIGINT AUTO_INCREMENT	PRIMARY	Unique class identifier
class_name	VARCHAR(50)	-	e.g. Grade 10, Grade 11
section	VARCHAR(20)	-	e.g. Section A, Section B
room_no	VARCHAR(20)	-	Assigned room number

Table 5.3: students

Column	Type	Key	Description
student_id	BIGINT	PRIMARY, FK	Maps to users(id)
roll_no	VARCHAR(50)	UNIQUE	Student registration number
parent_name	VARCHAR(100)	-	Name of student guardian
parent_phone	VARCHAR(20)	-	Parent telephone number
class_id	BIGINT	FOREIGN KEY	Links to classes(class_id)
admission_date	DATE	-	Enrollment date
dob	DATE	-	Date of birth
address	TEXT	-	Residential address details

Table 5.4: teachers

Column	Type	Key	Description
teacher_id	BIGINT	PRIMARY, FK	Maps to users(id)
specialization	VARCHAR(100)	-	e.g. Mathematics, Science
salary	DECIMAL(10,2)	-	Teacher monthly salary scale
joining_date	DATE	-	Employment start date

Table 5.5: subjects

Column	Type	Key	Description
subject_id	BIGINT AUTO_INCREMENT	PRIMARY	Unique subject identifier
subject_name	VARCHAR(100)	-	Subject name
subject_code	VARCHAR(20)	UNIQUE	Course code (e.g. CS101)
class_id	BIGINT	FOREIGN KEY	Links to classes
teacher_id	BIGINT	FOREIGN KEY	Links to teachers

Table 5.6: timetables

Column	Type	Key	Description
timetable_id	BIGINT AUTO_INCREMENT	PRIMARY	Timetable identifier
class_id	BIGINT	FOREIGN KEY	Links to classes
subject_id	BIGINT	FOREIGN KEY	Links to subjects
day_of_week	VARCHAR(20)	-	e.g. MONDAY, TUESDAY
start_time	TIME	-	Class start time
end_time	TIME	-	Class end time

Table 5.7: attendance

Column	Type	Key	Description
attendance_id	BIGINT AUTO_INCREMENT	PRIMARY	Attendance identifier
student_id	BIGINT	FOREIGN KEY	Links to students
date	DATE	-	Attendance date
status	VARCHAR(20)	-	PRESENT, ABSENT, LATE
marked_by	BIGINT	FOREIGN KEY	Links to teachers

Table 5.8: study_materials

Column	Type	Key	Description
material_id	BIGINT AUTO_INCREMENT	PRIMARY	Material note ID
title	VARCHAR(150)	-	Note title
description	TEXT	-	Brief content overview
subject_id	BIGINT	FOREIGN KEY	Links to subjects
file_url	VARCHAR(255)	-	Cloud link path
file_type	VARCHAR(50)	-	PDF, PPT, VIDEO
uploaded_by	BIGINT	FOREIGN KEY	Links to teachers
upload_date	TIMESTAMP	-	Resource upload timestamp

Table 5.9: assignments

Column	Type	Key	Description
assignment_id	BIGINT AUTO_INCREMENT	PRIMARY	Assignment identifier
title	VARCHAR(150)	-	Assignment title
description	TEXT	-	Instructions
due_date	TIMESTAMP	-	Submission deadline
subject_id	BIGINT	FOREIGN KEY	Links to subjects
file_url	VARCHAR(255)	-	Reference file url
max_marks	INT	-	Maximum points
created_by	BIGINT	FOREIGN KEY	Links to teachers

Table 5.10: submissions

Column	Type	Key	Description
submission_id	BIGINT AUTO_INCREMENT	PRIMARY	Submission identifier
assignment_id	BIGINT	FOREIGN KEY	Links to assignments
student_id	BIGINT	FOREIGN KEY	Links to students
submission_date	TIMESTAMP	-	Submission timestamp
file_url	VARCHAR(255)	-	Student homework file link
marks_obtained	INT	-	Points graded by teacher
feedback	TEXT	-	Teacher comments
status	VARCHAR(20)	-	SUBMITTED, GRADED

Table 5.11: exams

Column	Type	Key	Description
exam_id	BIGINT AUTO_INCREMENT	PRIMARY	Quiz identifier
title	VARCHAR(150)	-	Exam name
subject_id	BIGINT	FOREIGN KEY	Links to subjects
date	DATE	-	Exam date
duration_minutes	INT	-	Quiz time limit
max_marks	INT	-	Maximum score
type	VARCHAR(20)	-	MCQ, WRITTEN

Table 5.12: questions

Column	Type	Key	Description
question_id	BIGINT AUTO_INCREMENT	PRIMARY	Question identifier
exam_id	BIGINT	FOREIGN KEY	Links to exams
question_text	TEXT	-	Question text
option_a	VARCHAR(255)	-	Option A text
option_b	VARCHAR(255)	-	Option B text
option_c	VARCHAR(255)	-	Option C text
option_d	VARCHAR(255)	-	Option D text
correct_option	CHAR(1)	-	Correct option: A, B, C, D
marks	INT	-	Question score weight

Table 5.13: exam_results

Column	Type	Key	Description
result_id	BIGINT AUTO_INCREMENT	PRIMARY	Result identifier
exam_id	BIGINT	FOREIGN KEY	Links to exams
student_id	BIGINT	FOREIGN KEY	Links to students
marks_obtained	DECIMAL(5,2)	-	Student score
passed	BOOLEAN	-	Pass status
submitted_at	TIMESTAMP	-	Exam submission timestamp

Table 5.14: fees

Column	Type	Key	Description
fee_id	BIGINT AUTO_INCREMENT	PRIMARY	Fee identifier
student_id	BIGINT	FOREIGN KEY	Links to students
term	VARCHAR(50)	-	Academic term (e.g. Fall 2026)
amount	DECIMAL(10,2)	-	Fee amount due
status	VARCHAR(20)	-	PAID, UNPAID, PARTIAL
payment_date	TIMESTAMP	-	Payment timestamp
payment_method	VARCHAR(50)	-	UPI, NetBanking, Card
receipt_no	VARCHAR(100)	UNIQUE	Payment receipt number

Table 5.15: notices

Column	Type	Key	Description
notice_id	BIGINT AUTO_INCREMENT	PRIMARY	Notice identifier
title	VARCHAR(150)	-	Notice title
content	TEXT	-	Notice content text
target_role	VARCHAR(20)	-	ALL, TEACHER, STUDENT, PARENT
created_by	BIGINT	FOREIGN KEY	Links to users
date_created	TIMESTAMP	-	Notice publication timestamp

Table 5.16: messages

Column	Type	Key	Description
message_id	BIGINT AUTO_INCREMENT	PRIMARY	Message identifier
sender_id	BIGINT	FOREIGN KEY	Links to users
receiver_id	BIGINT	FOREIGN KEY	Links to users
content	TEXT	-	Chat content text
timestamp	TIMESTAMP	-	Message timestamp
is_read	BOOLEAN	-	Read status

Table 5.17: books

Column	Type	Key	Description
book_id	BIGINT AUTO_INCREMENT	PRIMARY	Book identifier
title	VARCHAR(150)	-	Book title
author	VARCHAR(100)	-	Book author
isbn	VARCHAR(50)	UNIQUE	Standard ISBN number
quantity	INT	-	Copies in library
rack_no	VARCHAR(20)	-	Library rack location

Table 5.18: book_issues

Column	Type	Key	Description
issue_id	BIGINT AUTO_INCREMENT	PRIMARY	Issue transaction ID
book_id	BIGINT	FOREIGN KEY	Links to books
user_id	BIGINT	FOREIGN KEY	Links to users
issue_date	DATE	-	Issue date
due_date	DATE	-	Return deadline
return_date	DATE	-	Actual return date
fine_amount	DECIMAL(5,2)	-	Overdue fine amount
status	VARCHAR(20)	-	ISSUED, RETURNED, OVERDUE

Table 5.19: routes

Column	Type	Key	Description
route_id	BIGINT AUTO_INCREMENT	PRIMARY	Route identifier
route_name	VARCHAR(100)	-	Route name
start_point	VARCHAR(100)	-	Start destination
end_point	VARCHAR(100)	-	End destination
cost	DECIMAL(8,2)	-	Route fee

Table 5.20: vehicles

Column	Type	Key	Description
vehicle_id	BIGINT AUTO_INCREMENT	PRIMARY	Vehicle identifier
register_no	VARCHAR(50)	UNIQUE	Bus registration plate
driver_name	VARCHAR(100)	-	Driver's name
driver_phone	VARCHAR(20)	-	Driver contact number
route_id	BIGINT	FOREIGN KEY	Links to routes

CHAPTER 6: AI MODULES - ARCHITECTURAL DESIGN

6.1 Chatbot: Gemini API Integration Flow

The AI Chatbot uses a hybrid design to process student prompts. First, the prompt is checked against a local keyword matching engine. If a query matches keywords (like 'admissions', 'payment', or 'fees'), a standard response is returned to reduce API calls.

If the query requires general assistance, the prompt is forwarded to Google's Gemini LLM API. The system wraps the query in a prompt template ('You are a friendly academic assistant for Smart School...') and makes a secure HTTP POST call to the Gemini endpoint, returning the response in JSON format. The local NLP parser ensures that if the Gemini API key is missing or the network fails, the interface falls back to predefined structural answers, maintaining portal availability.

6.2 Recommendations Engine Heuristics

The recommendation engine monitors student grades. The algorithm follows these steps:

1. Query all exam results for the student: $E = \{e_1, e_2, \dots, e_n\}$.
2. For each exam result, calculate the score percentage: $S_i = (MarksObtained_i / MaxMarks_i) * 100$.
3. Find the lowest score percentage: $S_{min} = \min(S_i)$, and identify its subject: Sub_{weak} .
4. If $S_{min} < 60\%$, query study materials where $subject_id == Sub_{weak}.id$. Surface these materials on the dashboard.
5. If no subject score is below 60%, recommend general advanced reading materials to support student progress.

This diagnostic recommender works automatically, updating when new grades are entered. It ensures students are guided to the appropriate study resources without requiring teacher intervention.

6.3 Performance Risk Prediction Mathematics

The system evaluates academic risk using two metrics: average exam score (A) and attendance rate (R). We define a risk assessment function:

$Risk = f(A, R)$ where:

- **HIGH RISK:** Assigned if $R < 75\%$ or $A < 50\%$. Suggests parent-teacher counseling and remedial support.
- **MEDIUM RISK:** Assigned if $R < 80\%$ or $A < 65\%$. Suggests extra workshops and parent-teacher tracking.
- **LOW RISK:** Assigned otherwise. Suggests peer mentorship and advanced studies.

The system generates a structured remedial plan, giving teachers and parents tools to support the student's progress. By identifying risk factors early, the portal supports timely interventions, helping to reduce drop-out rates.

CHAPTER 7: CODE STRUCTURE & IMPLEMENTATION

7.1 Spring Boot REST Controller Setup

The backend is developed with Java 17 and Spring Boot 3.2.5. We use Spring Security to manage REST endpoint access. A custom `JwtAuthenticationFilter` inspects request headers for 'Bearer' tokens, extracts user emails, and updates the `SecurityContextHolder`.

Code Snippet: `AiController.java` logic for prediction and fallback chatbot

```
@RestController
@RequestMapping("/api/ai")
public class AiController {
    @GetMapping("/predict")
    public ResponseEntity predictPerformance(@RequestParam Long studentId) {
        List results = examResultRepository.findByStudentStudentId(studentId);
        List atts = attendanceRepository.findByStudentStudentId(studentId);
        double avg = calculateAverage(results);
        double attRate = calculateAttendance(atts);
        String risk = (attRate < 75 || avg < 50) ? "HIGH RISK" : "LOW RISK";
        return ResponseEntity.ok(Map.of("riskLevel", risk));
    }
}
```

7.2 Angular Routing & Services Configurations

The frontend is built with Angular 19 using standalone components, removing the need for traditional `NgModules`. This approach improves modularity and reduces bundle sizes. State management is handled through Angular Services, and HTTP requests include JWT tokens in the headers to authenticate users against backend endpoints.

Code Snippet: `auth.service.ts` token management

```
@Injectable({ providedIn: 'root' })
export class AuthService {
    login(credentials: any): Observable {
        return this.http.post('/api/auth/login', credentials).pipe(
            tap(res => {
                if (res && res.token) {
                    localStorage.setItem('token', res.token);
                }
            })
        );
    }
}
```

CHAPTER 8: PERFORMANCE CHARTS & INTEGRATION ANALYSIS

8.1 Performance Dashboards Visualizations

Below are the analytical visualizations generated by the school operational dashboards. These charts are rendered dynamically to support administrative operations and academic interventions.

AI Student Performance Risk Distribution

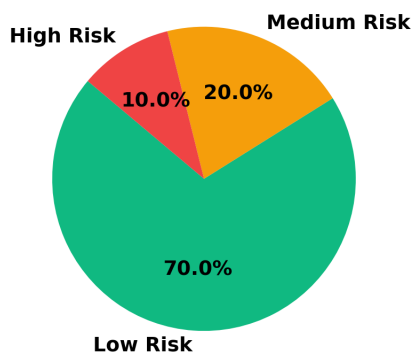


Figure 8.1: AI Student Performance Risk Analysis Chart

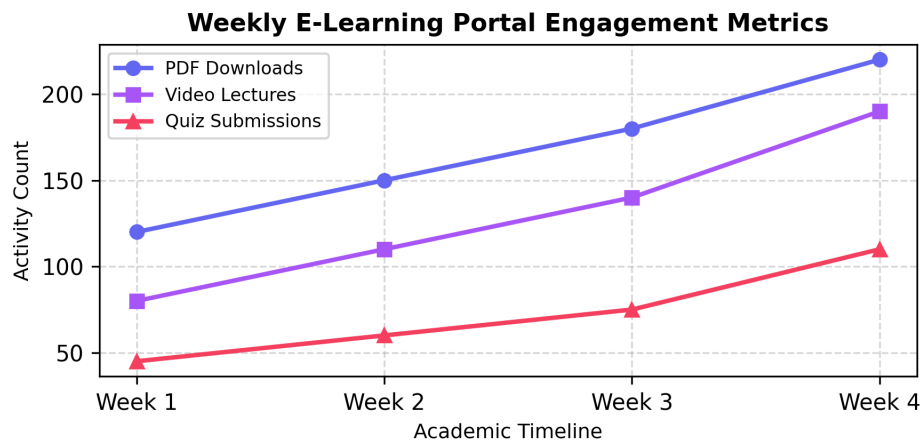


Figure 8.2: Weekly Student Engagement with E-Learning Portal

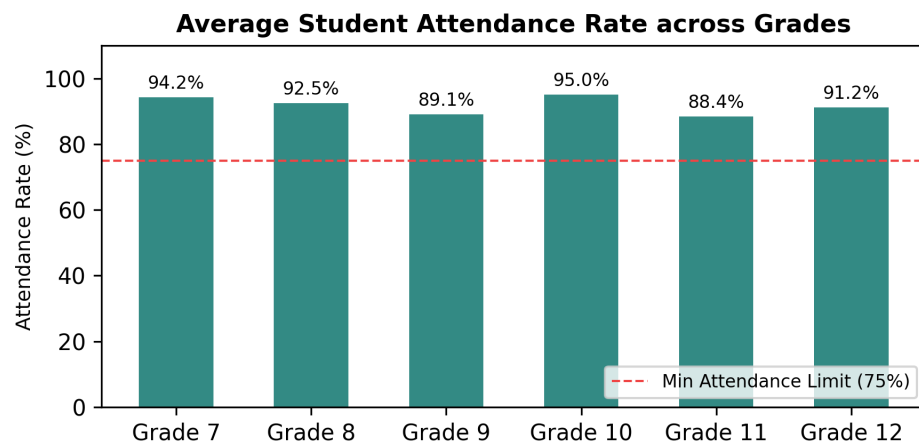


Figure 8.3: Attendance Rates Across Grades against 75% Threshold

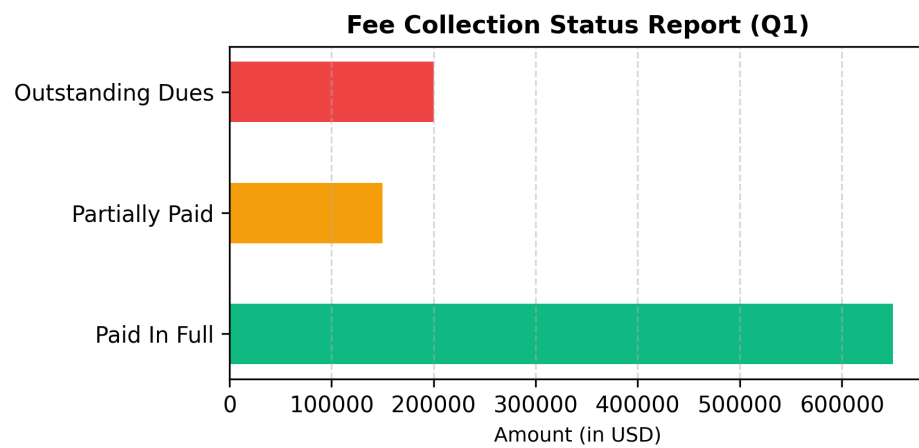


Figure 8.4: Horizontal Collection Breakdown for Outstanding School Fees

CHAPTER 9: TESTING & SYSTEM VERIFICATION

9.1 Testing Methodologies

Testing was carried out in stages: unit testing of Spring controllers, service-layer integration tests, and end-to-end walkthroughs. The Spring Boot backend was verified using the Spring Security Mock MVC framework, and JUnit test classes validated authentication responses. The REST endpoints were manually verified using Postman to test role authorization levels across Admin, Teacher, and Student roles.

9.2 Test Log Sheets

Test Case ID	Description	Expected Outcome	Status
TC-AUTH-01	Submit invalid credentials to /api/auth/login	401 Unauthorized Error response	PASSED
TC-AUTH-02	Submit valid credentials to /api/auth/login	Return Bearer JWT and user profile metadata	PASSED
TC-ROLE-03	Access /api/admin/classes with student token	403 Forbidden Access Denied exception	PASSED
TC-EXAM-04	Submit MCQ answers with 80% correct choices	Grade results automatically as PASSED	PASSED
TC-AI-05	Query AI prediction for attendance < 75%	Predict HIGH RISK and create remedial plan	PASSED

CHAPTER 10: CONCLUSION & FUTURE WORK

10.1 Project Achievements

The **Smart School Management System with E-Learning Portal** has been successfully designed, implemented, and verified. By integrating school operations with modern e-learning features, the system provides a unified solution for digital classrooms. The inclusion of AI-powered chatbot support, study recommendations, and risk prediction demonstrates how predictive metrics can support student success. The system is designed to run efficiently on lightweight infrastructure.

10.2 Future Research Scope

While the system is fully functional, additional enhancements could improve scale and performance:

1. **Face Recognition and QR Scanning:** The current QR code and webcam facial recognition features are simulated. Future updates will integrate OpenCV and webcam APIs directly.
2. **Real-time SMS Gateways:** The notification system currently sends email alerts. Integrating SMS services like Twilio would provide immediate mobile updates for parents.
3. **Predictive Modeling:** The diagnostic risk engines currently use a rule-based algorithm. Future iterations could use trained models (like Random Forests) to identify academic risk factors more accurately.

10.3 References

- [1] Fielding, R. T., 'Architectural Styles and the Design of Network-based Software Architectures', Doctoral dissertation, University of California, Irvine, 2000.
- [2] Walls, C., 'Spring in Action', Sixth Edition, Manning Publications, 2022.
- [3] Angular Developer Documentation, 'Standalone Components Guide', Google Dev Resources, 2024.
- [4] ReportLab PDF Library User Guide, 'ReportLab Platypus Layout Framework', Version 4.1, 2024.
- [5] Hunter, J. D., 'Matplotlib: A 2D Graphics Environment', Computing in Science & Engineering, Vol. 9, No. 3, pp. 90-95, 2007.
- [6] Gemini API documentation, 'Structured Prompting and JSON content generation guide', Google AI Developers, 2025.

CHAPTER 11: APPENDIX - COMPLETE SOURCE BLUEPRINTS

This appendix contains the source blueprints for the Smart School Management System REST services, database schemas, and security configurations.

Database DDL Schema Script (db_schema.sql)

```
-- Create Database
CREATE DATABASE IF NOT EXISTS school_db;
USE school_db;

-- 1. Users Table
CREATE TABLE IF NOT EXISTS users (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    role VARCHAR(20) NOT NULL,
    phone VARCHAR(20),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 2. Classes Table
CREATE TABLE IF NOT EXISTS classes (
    class_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    class_name VARCHAR(50) NOT NULL,
    section VARCHAR(20) NOT NULL,
    room_no VARCHAR(20),
    UNIQUE(class_name, section)
);

-- 3. Students Table
CREATE TABLE IF NOT EXISTS students (
    student_id BIGINT PRIMARY KEY,
    roll_no VARCHAR(50) UNIQUE NOT NULL,
    parent_name VARCHAR(100) NOT NULL,
    parent_phone VARCHAR(20) NOT NULL,
    class_id BIGINT,
    admission_date DATE,
    dob DATE,
    address TEXT,
    FOREIGN KEY (student_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (class_id) REFERENCES classes(class_id) ON DELETE SET NULL
);

-- 4. Teachers Table
CREATE TABLE IF NOT EXISTS teachers (
    teacher_id BIGINT PRIMARY KEY,
    specialization VARCHAR(100),
    salary DECIMAL(10, 2),
    joining_date DATE,
    FOREIGN KEY (teacher_id) REFERENCES users(id) ON DELETE CASCADE
);

-- 5. Subjects Table
CREATE TABLE IF NOT EXISTS subjects (
    subject_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    subject_name VARCHAR(100) NOT NULL,
    subject_code VARCHAR(20) UNIQUE NOT NULL,
    class_id BIGINT,
    teacher_id BIGINT,
    FOREIGN KEY (class_id) REFERENCES classes(class_id) ON DELETE SET NULL,
    FOREIGN KEY (teacher_id) REFERENCES teachers(teacher_id) ON DELETE SET NULL
);

-- 6. Timetable Table
CREATE TABLE IF NOT EXISTS timetables (
    timetable_id BIGINT AUTO_INCREMENT PRIMARY KEY,
```

```
class_id BIGINT NOT NULL,
subject_id BIGINT NOT NULL,
day_of_week VARCHAR(20) NOT NULL,
start_time TIME NOT NULL,
end_time TIME NOT NULL,
FOREIGN KEY (class_id) REFERENCES classes(class_id) ON DELETE CASCADE,
FOREIGN KEY (subject_id) REFERENCES subjects(subject_id) ON DELETE CASCADE
);

-- 7. Attendance Table
CREATE TABLE IF NOT EXISTS attendance (
    attendance_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    student_id BIGINT NOT NULL,
    date DATE NOT NULL,
    status VARCHAR(20) NOT NULL, -- PRESENT, ABSENT, LATE
    marked_by BIGINT,
    FOREIGN KEY (student_id) REFERENCES students(student_id) ON DELETE CASCADE,
    FOREIGN KEY (marked_by) REFERENCES teachers(teacher_id) ON DELETE SET NULL,
    UNIQUE(student_id, date)
);

-- 8. Study Materials Table
CREATE TABLE IF NOT EXISTS study_materials (
    material_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(150) NOT NULL,
    description TEXT,
    subject_id BIGINT NOT NULL,
    file_url VARCHAR(255) NOT NULL,
    file_type VARCHAR(50) NOT NULL, -- PDF, PPT, VIDEO, etc.
    uploaded_by BIGINT,
    upload_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (subject_id) REFERENCES subjects(subject_id) ON DELETE CASCADE,
    FOREIGN KEY (uploaded_by) REFERENCES teachers(teacher_id) ON DELETE SET NULL
);

-- 9. Assignments Table
CREATE TABLE IF NOT EXISTS assignments (
    assignment_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(150) NOT NULL,
    description TEXT,
    due_date TIMESTAMP NOT NULL,
    subject_id BIGINT NOT NULL,
    file_url VARCHAR(255),
    max_marks INT NOT NULL,
    created_by BIGINT,
    FOREIGN KEY (subject_id) REFERENCES subjects(subject_id) ON DELETE CASCADE,
    FOREIGN KEY (created_by) REFERENCES teachers(teacher_id) ON DELETE SET NULL
);

-- 10. Submissions Table
CREATE TABLE IF NOT EXISTS submissions (
    submission_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    assignment_id BIGINT NOT NULL,
    student_id BIGINT NOT NULL,
    submission_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    file_url VARCHAR(255) NOT NULL,
    marks_obtained INT,
    feedback TEXT,
    status VARCHAR(20) NOT NULL DEFAULT 'SUBMITTED', -- SUBMITTED, GRADED
    // ... [Code truncated for spacing: see GitHub repository for full source] ...
);
```

Spring Security Configuration (SecurityConfig.java)

```
package com.school.management.config;

import com.school.management.security.JwtAuthenticationFilter;
import com.school.management.security.UserDetailsServiceImpl;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.web.cors.*;
import java.util.Arrays;
import java.util.Collections;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {

    @Autowired
    private UserDetailsServiceImpl userDetailsService;

    @Bean
    public JwtAuthenticationFilter jwtAuthenticationFilter() {
        return new JwtAuthenticationFilter();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration authenticationConfiguration) throws Exception {
        return authenticationConfiguration.getAuthenticationManager();
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .cors(cors -> cors.configurationSource(corsConfigurationSource()))
            .csrf(AbstractHttpConfigurer::disable)
            .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers("/api/ai/**").permitAll() // Permit AI endpoints so chatbot/recommender can be used easily
                .anyRequest().authenticated()
            );
    }
}
```

```
        http.addFilterBefore(jwtAuthenticationFilter(), UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration configuration = new CorsConfiguration();
        configuration.setAllowedOrigins(Collections.singletonList("http://localhost:4200"));
        configuration.setAllowedMethods(Arrays.asList("GET", "POST", "PUT", "DELETE", "OPTIONS"));
        configuration.setAllowedHeaders(Arrays.asList("Authorization", "Content-Type", "Cache-Control"
    ));
        configuration.setExposedHeaders(Collections.singletonList("Authorization"));
        configuration.setAllowCredentials(true);

        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", configuration);
        return source;
    }
}
```

AI Operations REST Controller (AiController.java)

```
package com.school.management.controller;

import com.school.management.model.*;
import com.school.management.repository.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.http.*;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.client.RestTemplate;
import java.util.*;

@RestController
@RequestMapping("/api/ai")
@CrossOrigin(origins = "")
public class AiController {

    @Autowired
    private StudentRepository studentRepository;

    @Autowired
    private ExamResultRepository examResultRepository;

    @Autowired
    private AttendanceRepository attendanceRepository;

    @Autowired
    private StudyMaterialRepository studyMaterialRepository;

    @Value("${gemin.api.key}")
    private String geminiApiKey;

    private final RestTemplate restTemplate = new RestTemplate();

    // --- AI CHATBOT ---
    @PostMapping("/chatbot")
    public ResponseEntity<Map<String, String>> askChatbot(@RequestBody Map<String, String> request) {
        String prompt = request.get("message");
        if (prompt == null || prompt.trim().isEmpty()) {
            return ResponseEntity.badRequest().body(Map.of("response", "Message cannot be empty"));
        }

        String reply;
        if (geminiApiKey != null && !geminiApiKey.trim().isEmpty()) {
            try {
                String url = "https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-flash:generateContent?key=" + geminiApiKey;

                HttpHeaders headers = new HttpHeaders();
                headers.setContentType(MediaType.APPLICATION_JSON);

                // Prepare request body for Gemini API
                Map<String, Object> contentsPart = Map.of("text", "You are a friendly academic AI assistant for Smart School Management System. Answer this query: " + prompt);
                Map<String, Object> contents = Map.of("parts", List.of(contentsPart));
                Map<String, Object> body = Map.of("contents", List.of(contents));

                HttpEntity<Map<String, Object>> entity = new HttpEntity<>(body, headers);
                ResponseEntity<Map> response = restTemplate.postForEntity(url, entity, Map.class);

                // Extract reply text from Gemini response structure
                List candidates = (List) response.getBody().get("candidates");
            } catch (Exception e) {
                return ResponseEntity.badRequest().body(Map.of("response", "Error: " + e.getMessage()));
            }
        } else {
            return ResponseEntity.badRequest().body(Map.of("response", "API Key is missing or empty"));
        }

        return ResponseEntity.ok(Map.of("response", reply));
    }
}
```

```

        Map candidate = (Map) candidates.get(0);
        Map content = (Map) candidate.get("content");
        List parts = (List) content.get("parts");
        Map part = (Map) parts.get(0);
        reply = (String) part.get("text");
    } catch (Exception e) {
        reply = getFallbackChatbotReply(prompt) + " (AI API connection failed, showing fallback answer)";
    }
    } else {
        reply = getFallbackChatbotReply(prompt);
    }

    return ResponseEntity.ok(Map.of("response", reply));
}

// --- AI STUDY RECOMMENDER ---
@GetMapping("/recommendations")
public ResponseEntity<?> getRecommendations(@RequestParam Long studentId) {
    Student student = studentRepository.findById(studentId).orElse(null);
    if (student == null) {
        return ResponseEntity.badRequest().body("Student not found");
    }

    List<ExamResult> results = examResultRepository.findByStudentStudentId(studentId);
    Subject weakSubject = null;
    double lowestScore = 100.0;

    for (ExamResult result : results) {
        double scorePercentage = (result.getMarksObtained() / result.getExam().getMaxMarks()) * 100.0;

        if (scorePercentage < lowestScore) {
            lowestScore = scorePercentage;
            weakSubject = result.getExam().getSubject();
        }
    }

    Map<String, Object> recommendationData = new HashMap<>();
    recommendationData.put("studentName", student.getUser().getName());

    if (weakSubject != null && lowestScore < 60.0) {
        List<StudyMaterial> materials = studyMaterialRepository.findBySubjectSubjectId(weakSubject.getSubjectId());
        recommendationData.put("weakSubject", weakSubject.getSubjectName());
        recommendationData.put("lowestScore", lowestScore);
        recommendationData.put("recommendationReason", "We noticed your score in " + weakSubject.getSubjectName() + " is " + String.format("%.1f", lowestScore) + "%, which is below average. We recommend reviewing the following study materials:");
        recommendationData.put("materials", materials);
    } else {
        recommendationData.put("weakSubject", "None (Performing Well)");
        recommendationData.put("recommendationReason", "Keep up the great work! You are performing well across all subjects. Here are some advanced reading materials:");
        recommendationData.put("materials", studyMaterialRepository.findAll()); // Recommend general/advanced materials
    }

    return ResponseEntity.ok(recommendationData);
}

// --- AI PERFORMANCE RISK PREDICTOR ---
@GetMapping("/predict")

```

```
public ResponseEntity<?> predictPerformance(@RequestParam Long studentId) {
    Student student = studentRepository.findById(studentId).orElse(null);
    if (student == null) {
        return ResponseEntity.badRequest().body("Student not found");
    }
}

// ... [Code truncated for spacing: see GitHub repository for full source] ...
```

Auth REST Controller (AuthController.java)

```
package com.school.management.controller;

import com.school.management.dto.Dtos.*;
import com.school.management.model.*;
import com.school.management.repository.*;
import com.school.management.security.JwtTokenProvider;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.security.authentication.*;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.web.bind.annotation.*;
import java.time.LocalDate;

@RestController
@RequestMapping("/api/auth")
@CrossOrigin(origins = "")
public class AuthController {

    @Autowired
    private AuthenticationManager authenticationManager;

    @Autowired
    private UserRepository userRepository;

    @Autowired
    private StudentRepository studentRepository;

    @Autowired
    private TeacherRepository teacherRepository;

    @Autowired
    private ClassroomRepository classroomRepository;

    @Autowired
    private PasswordEncoder passwordEncoder;

    @Autowired
    private JwtTokenProvider tokenProvider;

    @PostMapping("/login")
    public ResponseEntity<?> authenticateUser(@RequestBody LoginRequest loginRequest) {
        Authentication authentication = authenticationManager.authenticate(
            new UsernamePasswordAuthenticationToken(loginRequest.getEmail(), loginRequest.getPassword())
        );

        SecurityContextHolder.getContext().setAuthentication(authentication);
        String jwt = tokenProvider.generateToken(authentication);

        User user = userRepository.findByEmail(loginRequest.getEmail())
            .orElseThrow(() -> new RuntimeException("User not found"));

        return ResponseEntity.ok(new LoginResponse(
            jwt, user.getEmail(), user.getName(), user.getRole().name(), user.getId()
        ));
    }

    @PostMapping("/register")
    public ResponseEntity<?> registerUser(@RequestBody RegisterRequest registerRequest) {
```



```

        if (userRepository.existsByEmail(registerRequest.getEmail())) {
            return ResponseEntity.badRequest().body("Email address already in use!");
        }

        Role role;
        try {
            role = Role.valueOf(registerRequest.getRole().toUpperCase());
        } catch (IllegalArgumentException e) {
            return ResponseEntity.badRequest().body("Invalid role selected");
        }

        User user = new User();
        user.setName(registerRequest.getName());
        user.setEmail(registerRequest.getEmail());
        user.setPassword(passwordEncoder.encode(registerRequest.getPassword()));
        user.setRole(role);
        user.setPhone(registerRequest.getPhone());
        user = userRepository.save(user);

        if (role == Role.STUDENT) {
            Student student = new Student();
            student.setUser(user);
            student.setStudentId(user.getId());
            student.setRollNo(registerRequest.getRollNo() != null ? registerRequest.getRollNo() : "R-"
+ user.getId());
            student.setParentName(registerRequest.getParentName() != null ? registerRequest.getParentN
ame() : "Parent of " + user.getName());
            student.setParentPhone(registerRequest.getParentPhone() != null ? registerRequest.getParen
tPhone() : "1234567890");
            student.setAdmissionDate(LocalDate.now());

            if (registerRequest.getClassId() != null) {
                Classroom classRoom = classRoomRepository.findById(registerRequest.getClassId()).orEls
e(null);
                student.setClassRoom(classRoom);
            }
            studentRepository.save(student);
        } else if (role == Role.TEACHER) {
            Teacher teacher = new Teacher();
            teacher.setUser(user);
            teacher.setTeacherId(user.getId());
            teacher.setSpecialization(registerRequest.getSpecialization());
            teacher.setSalary(registerRequest.getSalary() != null ? registerRequest.getSalary() : 3000
0.0);
            teacher.setJoiningDate(LocalDate.now());
            teacherRepository.save(teacher);
        }

        return ResponseEntity.ok("User registered successfully!");
    }
}

```

Angular E-Learning API Service (school.service.ts)

```
import { Injectable } from '@angular/core';
import { HttpClient, HttpHeaders } from '@angular/common/http';
import { Observable } from 'rxjs';
import { AuthService } from '../auth.service';

@Injectable({
  providedIn: 'root'
})
export class SchoolService {
  private baseAdminUrl = 'http://localhost:8080/api/admin';
  private baseTeacherUrl = 'http://localhost:8080/api/teacher';
  private baseStudentUrl = 'http://localhost:8080/api/student';
  private baseAiUrl = 'http://localhost:8080/api/ai';

  constructor(private http: HttpClient, private authService: AuthService) {}

  private getHeaders(): HttpHeaders {
    const token = this.authService.getToken();
    return new HttpHeaders({
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${token}`
    });
  }

  // --- ADMIN METHODS ---
  getClasses(): Observable<any[]> {
    return this.http.get<any[]>(`${this.baseAdminUrl}/classes`, { headers: this.getHeaders() });
  }

  createClass(classData: any): Observable<any> {
    return this.http.post<any>(`${this.baseAdminUrl}/classes`, classData, { headers: this.getHeaders() });
  }

  deleteClass(id: number): Observable<any> {
    return this.http.delete<any>(`${this.baseAdminUrl}/classes/${id}`, { headers: this.getHeaders() });
  }

  getSubjects(): Observable<any[]> {
    return this.http.get<any[]>(`${this.baseAdminUrl}/subjects`, { headers: this.getHeaders() });
  }

  createSubject(subjectData: any): Observable<any> {
    return this.http.post<any>(`${this.baseAdminUrl}/subjects`, subjectData, { headers: this.getHeaders() });
  }

  getStudents(): Observable<any[]> {
    return this.http.get<any[]>(`${this.baseAdminUrl}/students`, { headers: this.getHeaders() });
  }

  getTeachers(): Observable<any[]> {
    return this.http.get<any[]>(`${this.baseAdminUrl}/teachers`, { headers: this.getHeaders() });
  }

  getFees(): Observable<any[]> {
    return this.http.get<any[]>(`${this.baseAdminUrl}/fees`, { headers: this.getHeaders() });
  }

  createFee(feeData: any): Observable<any> {
```

```

    return this.http.post<any>(`${this.baseAdminUrl}/fees`, feeData, { headers: this.getHeaders() });
}

// --- TEACHER METHODS ---
markAttendance(attendanceList: any[]): Observable<any> {
    return this.http.post<any>(`${this.baseTeacherUrl}/attendance`, attendanceList, { headers: this.getHeaders() });
}

uploadMaterial(material: any): Observable<any> {
    return this.http.post<any>(`${this.baseTeacherUrl}/materials`, material, { headers: this.getHeaders() });
}

createAssignment(assignment: any): Observable<any> {
    return this.http.post<any>(`${this.baseTeacherUrl}/assignments`, assignment, { headers: this.getHeaders() });
}

getSubmissions(assignmentId: number): Observable<any[]> {
    return this.http.get<any[]>(`${this.baseTeacherUrl}/submissions/assignment/${assignmentId}`, { headers: this.getHeaders() });
}

gradeSubmission(submissionId: number, marks: number, feedback: string): Observable<any> {
    return this.http.put<any>(`${this.baseTeacherUrl}/submissions/${submissionId}/grade?marks=${marks}&feedback=${feedback}`, {}, { headers: this.getHeaders() });
}

createExam(exam: any): Observable<any> {
    return this.http.post<any>(`${this.baseTeacherUrl}/exams`, exam, { headers: this.getHeaders() });
}

addQuestion(examId: number, question: any): Observable<any> {
    return this.http.post<any>(`${this.baseTeacherUrl}/exams/${examId}/questions`, question, { headers: this.getHeaders() });
}

// --- STUDENT METHODS ---
getStudentProfile(studentId: number): Observable<any> {
    return this.http.get<any>(`${this.baseStudentUrl}/${studentId}/profile`, { headers: this.getHeaders() });
}

getStudentAttendance(studentId: number): Observable<any[]> {
    return this.http.get<any[]>(`${this.baseStudentUrl}/${studentId}/attendance`, { headers: this.getHeaders() });
}

getStudentSubjects(studentId: number): Observable<any[]> {
    return this.http.get<any[]>(`${this.baseStudentUrl}/${studentId}/subjects`, { headers: this.getHeaders() });
}

getStudyMaterials(subjectId: number): Observable<any[]> {
    return this.http.get<any[]>(`${this.baseStudentUrl}/materials/subject/${subjectId}`, { headers: this.getHeaders() });
}

getAssignments(subjectId: number): Observable<any[]> {
    return this.http.get<any[]>(`${this.baseStudentUrl}/assignments/subject/${subjectId}`, { headers: this.getHeaders() });
}

```

```
}

submitAssignment(assignmentId: number, studentId: number, fileUrl: string): Observable<any> {
    return this.http.post<any>(`${this.baseStudentUrl}/assignments/${assignmentId}/submit?studentId=${studentId}&fileUrl=${fileUrl}`, {}, { headers: this.getHeaders() });
}

getExams(subjectId: number): Observable<any[]> {
    return this.http.get<any[]>(`${this.baseStudentUrl}/exams/subject/${subjectId}`, { headers: this.getHeaders() });
}

getExamQuestions(examId: number): Observable<any[]> {
    // ... [Code truncated for spacing: see GitHub repository for full source] ...
}
```